

פתרון שאלה 1

סעיף א

```
public class Menu <E extends Dish> implements Iterable<E> {  
    private ArrayList<E> list = new ArrayList<E>();  
    public void add(E e) {  
        list.add(e);  
    }  
    public E get(String name) {  
        for (E e : list) {  
            if (e.getName().equals(name))  
                return e;  
        }  
        return null;  
    }  
    public Iterator<E> iterator() {  
        return list.iterator();  
    }  
}
```

סעיף ב

```
public static <E extends Dish> Menu<Cake> getCakes(Menu<E> m) {  
    Menu<Cake> cakes = new Menu<Cake>();  
    Iterator<E> it = m.iterator();  
    while(it.hasNext()) {  
        E next = it.next();  
        if (next instanceof Cake)  
            cakes.add((Cake)next);  
    }  
    return cakes;  
}
```

ישנם שתי אפשרויות לפיתרון: שימוש ב- 4 רשימות (אחת לכל קטגוריה)

```
public Iterator<E> iterator() {  
    ArrayList<E> first = new ArrayList<>();  
    ArrayList<E> main = new ArrayList<>();  
    ArrayList<E> beverage = new ArrayList<>();  
    ArrayList<E> desert = new ArrayList<>();  
    for(int i=0; i<list.size();i++) {  
        E item = list.get(i);  
        if(item.getCategory().equals("First"))  
            first.add(item);  
        else if(item.getCategory().equals("Main"))  
            main.add(item);  
        else if(item.getCategory().equals("Beverage"))  
            beverage.add(item);  
        else desert.add(item);  
    }  
    return new Iterator<>() {  
        public boolean hasNext() {  
            return !desert.isEmpty() || !beverage.isEmpty() ||  
                !main.isEmpty() || !first.isEmpty();  
        }  
        public E next() {  
            if(!first.isEmpty())  
                return first.remove(0);  
            else if(!main.isEmpty())  
                return main.remove(0);  
            else if(!beverage.isEmpty())  
                return beverage.remove(0);  
            return desert.remove(0);  
        }  
    }  
}
```

```

    }
};
}

```

שימוש בהעתק של הרשימה

```

public Iterator<E> iterator() {
    ArrayList<E> copy = new ArrayList<E>();
    copy.addAll(list);
    return new Iterator<>() {
        public boolean hasNext() {
            return !copy.isEmpty();
        }
        public E next() {
            for(int i=0; i<copy.size(); i++) {
                if(copy.get(i).getCategory().equals("First"))
                    return copy.remove(i);
            }
            for(int i=0; i<copy.size(); i++) {
                if(copy.get(i).getCategory().equals("Main"))
                    return copy.remove(i);
            }
            for(int i=0; i<copy.size(); i++) {
                if(copy.get(i).getCategory().equals("Beverage"))
                    return copy.remove(i);
            }
            return copy.remove(0);
        }
    };
}

```

סעיף ד 1

? extends Dish

המתודה main אינה גנרית ולכן במידה ורוצים ליצור Menu מסוגים שונים יש להשתמש ב-wildcard

סעיף ד 2

לא חוקי, אין קשר ירושה בין Main<Cake> ל- Main<Desert> למרות שהמחלקה Cake יורשת מ- Desert הירושה לא מתקיימת עבור הטיפוסים הגנריים. כדי שיהיה חוקי des צריך להיות מוגדר עם extend Desert ?

פתרון שאלה 2

1. `IDataStream`

2. `StreamingService`

3. `IConnectable`

4. שגיאת קומפילציה בגלל קריאה ל- `isLocal`

5. הקוד תקין

6. הקוד תקין ומהווה העמסה

7. שגיאת קומפילציה בגלל ההשמה ל- `a2`

8. הקוד תקין ומהווה דריסה

9. `AABABB`

10. `true,false,false, true`

```
public class ObjectManager{
    private Object[] objects;
    private Thread[] owners;
    public ObjectManager(int M){
        objects = new Object[M];
        for(int i=0; i<M; i++)
            objects[i] = new Object();
        owners = new Thread[M];
    }
    public synchronized Object getObject(int num){
        while(owners[num-1]!=null &&
            owners[num-1]!=Thread.currentThread() ){
            try{
                wait();
            }catch(InterruptedException e){ }
        }
        owners[num-1]=Thread.currentThread();
        return objects[num-1];
    }
    public synchronized void releaseObject(int num){
        if(owners[num-1]!=Thread.currentThread()) return;
        owners[num-1] = null;
        notifyAll();
    }
}
```

```
public class ObjectsUser implements Runnable{
    private ObjectManager manager;
    private List<Integer> nums;
    public ObjectsUser(ObjectManager m, List<Integer> nums){
        this.manager = m;
        this.nums = nums;
    }
    public void run(){
        List<Object> objects = new ArrayList<>();
        for(Integer num : nums){
            objects.add(m.getObject(num));
        }
        for(int i=0; i<objects.size(); i++){
            System.out.print(objects.get(i));
        }
        try{
            sleep(5000);
        }catch(InterruptedException e){ }
        for(Integer num : nums){
            m.releaseObject(num);
        }
    }
}
```